

WRO

JORNADAS DE ROBÓTICA, MAYO 2026 CIUDAD REAL COLEGIO NUESTRA SEÑORA DE LAS MERCEDES –HERENCIA-

Descripción técnica del robot:

Desarrollo: EL desarrollo del robot comenzó con una pequeña tabla de madera, creamos un prototipo y luego lo diseñamos en un programa de dibujo vectorial para fabricar el chasis con la cortadora láser.

Está diseñado a dos niveles, en el nivel inferior está el sistema de dirección, motor servo y baterías, y en el nivel superior los componentes electrónicos.

Componentes:

- Motor de corriente continua con reductora.
- L298N, para poder utilizar el motor.
- Placa microbit + Shield Keyestudio
- Portabaterías 18650 LIPO 2 en serie.
- Huskylens V1, cámara IA
- Interruptor
- Servo 180°.
- Ruedas Lego
- Sistema de cremallera + bielas fabricado con piezas de lego.
- Piezas diseñadas en 3D: engranajes, soportes y acoples.

Problemas: después de arreglar un problema del chasis, comenzamos a diseñar la planta superior en la que se encuentra la placa microbit, su expansión, el interruptor de encendido y el L298N, además de un soporte para que sujete el motor 180° de la parte inferior y mientras, en la parte inferior instalamos un portapilas para alimentar los componentes electrónicos y también el motor de corriente continua con reductora. Finalmente instalamos los pilares que mantienen unidas las dos partes y conectamos todos los componentes electrónicos con cables. Sin embargo, el interruptor se rompió por lo que tuvimos que cambiarlo al interruptor actual, por último imprimimos en una impresora 3D un soporte para mantener la cámara de IA huskylens más elevada que el resto de componentes y la conectamos a la expansión de la placa microbit. Tras terminar el robot, comenzamos a programar el robot para completar una serie de circuitos de obstáculos creados por nosotros mismos para encontrar y corregir errores de programación.

Decisiones técnicas:

- Uso de cámara IA, elevándola unos centímetros para controlar el horizonte.
- Usar microbit y tarjetas de expansión.
- Robot de tamaño pequeño.

Resultados obtenidos:

- Hemos conseguido un robot básico que no se mueve como nos hubiese gustado, pero que con más tiempo y dedicación podría haber funcionado muy bien.

Movilidad:

- Servomotor 180° para dirección, con ángulos +- 45°
- Motor de corriente continua con reductora para tracción trasera.

- Ruedas y chasis con piezas de lego.
- L298N para alimentar el motor DC.

Alimentación:

- 2 Pilas LiPo 18650

Sensores:

- Cámara IA huskylens

Código empleado

```
/**
 * para no tener que apagar y encender para empezar de nuevo al pulsar a después de acabar
 */
// Función para avanzar de forma continua sin pausas.
// Configura los pines de tracción hacia adelante y aplica la velocidad indicada.
function marchaContinua (velocidad: number) {
    // P2 a 0 y P8 a 1 determinan el sentido de giro del motor (Hacia adelante)
    pins.digitalWritePin(DigitalPin.P2, 0)
    pins.digitalWritePin(DigitalPin.P8, 1)
    // P1 regula la velocidad mediante una señal analógica (0 a 1023)
    pins.analogWritePin(AnalogPin.P1, velocidad)
}
// BUCLE
// Se ejecuta infinitamente tras pulsar el botón A.
input.onButtonPressed(Button.A, function () {
    while (true) {
        // 1. Condición de salida: Si la carrera terminó, detenemos todo.
        if (carreraTerminada) {
            parar(0)
            return
        }
    }
}
```

```

// 2. Lectura de la cámara HuskyLens
huskylens.request()
// Leemos el ID (Color) del objeto que está enfoca
cado
Color = huskylens.readBox_s(Content3.ID)
// Si detecta el color 4 (Meta), activa permanentemente el modo meta
if (Color == 4) {
    modoMeta = true
}
// =====
// (Buscando la meta - Color 4)
// =====
if (modoMeta) {
    // Muestra el icono de objetivo
    basic.showIcon(IconNames.Target)
    if (Color == 4) {
        // Leemos las coordenadas X e Y del centro del objeto
        posX = huskylens.readBox(Color, Content1.xCenter)
        posY = huskylens.readBox(Color, Content1.yCenter)

        // Lógica de dirección: Mantiene el objeto en el centro de la visión
        if (posX < 140) {
            // Gira para corregir a un lado
            servos.P0.setAngle(55)
        } else if (posX > 180) {
            // Gira para corregir al otro lado
            servos.P0.setAngle(135)
        } else {
            // Va recto si está centrado
            servos.P0.setAngle(90)
        }
    }
}

```

```

        // Acelera a máxima velocidad hacia la m
eta
        marchaContinua(1023)
        // Si la coordenada Y es mayor a 200, el
objeto está muy bajo en la pantalla,
        // lo que significa que lo tenemos justo
delante.
        if (posY > 200) {
            carreraTerminada = true
            parar(0)
            // Muestra una X al terminar
            basic.showIcon(IconNames.No)
        }
    } else {
        // Si perdemos de vista el color 4 :
        // Intentamos ir rectos medio segundo, p
or si lo encontramos.
        servos.P0.setAngle(90)
        marchaContinua(1023)
        basic.pause(500)
        // Si no reaparece, nos detenemos y term
inamos.
        carreraTerminada = true
        parar(0)
        basic.showIcon(IconNames.No)
    }
    return
}
// =====
// LÓGICA DE EVADIR OBSTÁCULOS (Obstáculos IDs 1
, 2 y 3)
// =====
if (Color > 0 && Color <= 3) {
    // Leemos posición Y TAMAÑO del obstáculo de
tectado

```

```

        posX = huskylens.readeBox(Color, Content1.xC
enter)
        posY = huskylens.readeBox(Color, Content1.yC
enter)
        ancho = huskylens.readeBox(Color, Content1.w
idth)
        alto = huskylens.readeBox(Color, Content1.he
ight)

        // --- DETECCIÓN DE CHOQUE ---
        // Se echa para atrás si:
        // 1. Está muy abajo en la pantalla (posY >
140), señal de que está muy cerca.
        // 2. 0 es muy ancho (ancho > 100), es decir
, es un gran muro.
        // 3. 0 es muy alto (alto > 100).
        if (posY > 140 || ancho > 100 || alto > 100)
{
            if (Color == 3) {
                // LÓGICA PARA PARED (Color 3)
                // Primero, da marcha atrás girando
hacia el lado opuesto del muro
                if (posX >= 160) {
                    servos.P0.setAngle(135)
                } else {
                    servos.P0.setAngle(55)
                }
                parar(150)
                // Marcha atrás durante 1 segundo
                atras(1023, 1000)
                parar(150)
                // Contravolante de salida: Cambia l
a dirección y avanza para salir en forma de "S"
                if (posX >= 160) {
                    servos.P0.setAngle(55)
                } else {

```

```

        servos.P0.setAngle(135)
    }
    adelante(750, 600)
    // Termina la maniobra de "S"
    if (posX >= 160) {
        servos.P0.setAngle(135)
    } else {
        servos.P0.setAngle(55)
    }
    adelante(750, 400)
} else {
    // LÓGICA PARA OBSTÁCULOS PEQUEÑOS (
IDs 1 y 2 cerca)
    // Da un pequeño toque marcha atrás
para alejarse

    if (Color == 1) {
        servos.P0.setAngle(55)
    } else {
        servos.P0.setAngle(135)
    }
    parar(150)
    atras(1023, 400)
}
// Después de maniobrar, pone las ruedas
rectas

    servos.P0.setAngle(90)
} else {
    // --
- AVANCE NORMAL / ESQUIVE DE LEJOS ---
    // Maniobra en "S" frontal, esquivando s
in tener que frenar ni dar marcha atrás
    if (Color == 1) {
        servos.P0.setAngle(135)
        adelante(750, 400)
        servos.P0.setAngle(55)
    }

```

```

        adelante(750, 400)
    } else if (Color == 2) {
        servos.P0.setAngle(55)
        adelante(750, 400)
        servos.P0.setAngle(135)
        adelante(750, 400)
    } else if (Color == 3) {
        // Si el obstáculo 3 está lejos, esq
uivamos hacia el lado más despejado
        if (posX >= 160) {
            servos.P0.setAngle(55)
            adelante(750, 400)
            servos.P0.setAngle(135)
            adelante(750, 400)
        } else {
            servos.P0.setAngle(135)
            adelante(750, 400)
            servos.P0.setAngle(55)
            adelante(750, 400)
        }
    }
    // Siempre ponemos la dirección al centr
o
    servos.P0.setAngle(90)
}
} else {
    // =====
=
    // NO VE NADA (Camino despejado)
    // =====
=
    // Ruedas rectas y avanza a velocidad modera
da para explorar
    servos.P0.setAngle(90)
    marchaContinua(750)

```

```

        }
    }
})
// Funciones de movimiento temporizado.
// Configuran la dirección del motor y duración movimiento.
function adelante (velocidad: number, tiempo: number) {
    pins.digitalWritePin(DigitalPin.P2, 0)
    pins.digitalWritePin(DigitalPin.P8, 1)
    pins.analogWritePin(AnalogPin.P1, velocidad)
    basic.pause(tiempo)
}
function parar (tiempo: number) {
    // Velocidad cero
    pins.analogWritePin(AnalogPin.P1, 0)
    basic.pause(tiempo)
}
function atras (velocidad: number, tiempo: number) {
    // Invertimos los pines lógicos P2 y P8 para invertir el giro del motor
    pins.digitalWritePin(DigitalPin.P2, 1)
    pins.digitalWritePin(DigitalPin.P8, 0)
    pins.analogWritePin(AnalogPin.P1, velocidad)
    basic.pause(tiempo)
}
let alto = 0
let ancho = 0
let posY = 0
let posX = 0
let modoMeta = false
let Color = 0
let carreraTerminada = false
// Se pone la cremallera de la dirección en posición inicial, apuntando recto
servos.P0.setAngle(90)

```



```
// Inicializar la cámara por el bus I2C y configurar el
// algoritmo de colores
huskylens.initI2c()
huskylens.initMode(protocolAlgorithm.ALGORITHM_COLOR_RECOGNITION)
// Preparar el controlador del motor para el avance (aun
// que no le da velocidad aún)
pins.digitalWritePin(DigitalPin.P2, 0)
pins.digitalWritePin(DigitalPin.P8, 1)
// Indicador visual de que el robot está listo para empe
// zar
basic.showIcon(IconNames.Happy)
carreraTerminada = false
```

Funcionamiento

El objetivo es que un robot pueda esquivar obstáculos según las instrucciones dadas. Para ello hemos programado el robot teniendo en cuenta dos cosas:

- Esquivar obstáculos (representados por los colores 1, 2 y 3)
- Aparcar (4). Objetivo no conseguido.
-

Para lograr esto, el código combina el control de un motor DC para la tracción, un servomotor para la dirección y la cámara HuskyLens para reconocer los objetos(colores)

Inicio:

El robot usa una dirección de tipo "cremallera" controlada por un servomotor conectado al pin P0. El ángulo de 90 grados representa tener las ruedas rectas. Los ángulos 55 y 135 se usan para girar a los lados.

La Tracción (Motor DC en P1, P2 y P8): El avance y retroceso se controlan mediante tres pines. P2 y P8 actúan como interruptores lógicos (0 o 1) para decidir si el motor gira hacia adelante o hacia atrás. El pin P1 envía una señal analógica (PWM) que regula la velocidad del motor, desde 0 (parado) hasta 1023 (máxima velocidad).

La Cámara (HuskyLens): Se inicializa la comunicación I2C con la cámara y se configura en el modo de "Reconocimiento de Colores".

Funciones de Movimiento

Hemos creado funciones auxiliares:

Adelante (velocidad, tiempo) y atrás (velocidad, tiempo), avanza o retrocede durante unos milisegundos

Parar. El motor DC deja de funcionar.

Bucle

Al pulsar el botón A, el robot entra en un bucle infinito.

Primero: comprueba si ha terminado el recorrido; si es así, detiene los motores y sale del bucle, terminando el programa.

Si no ha terminado, la cámara HuskyLens analiza lo que está viendo y obtiene el ID(el color) del objeto detectado en el centro de la pantalla. Dependiendo de lo que vea, el robot tomará una de tres decisiones: ir a aparcarse, esquivar un obstáculo o avanzar.

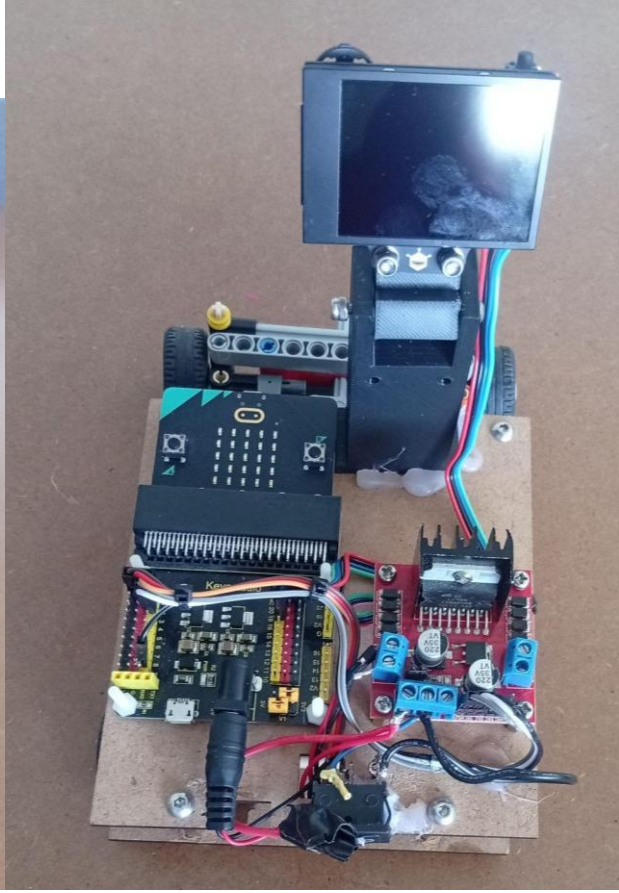
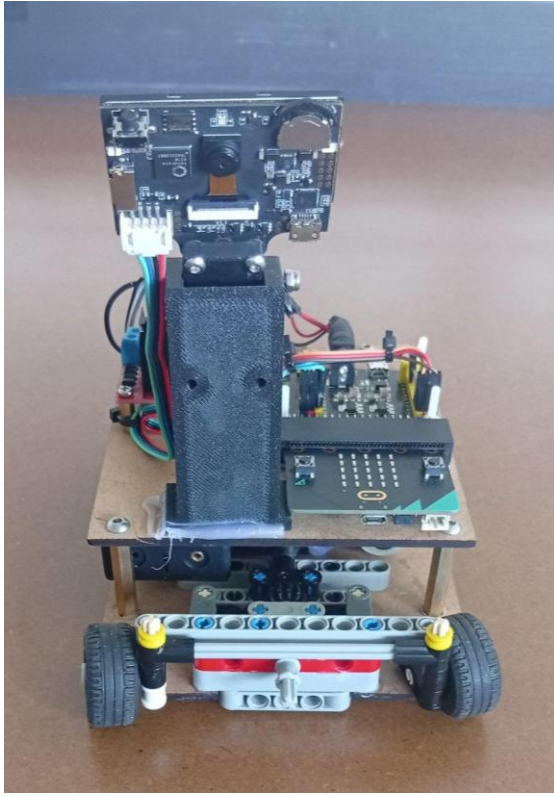
Si la cámara detecta el color con ID 4, el robot activa modoMeta = true, prioridad absoluta es alcanzar es aparcarse.

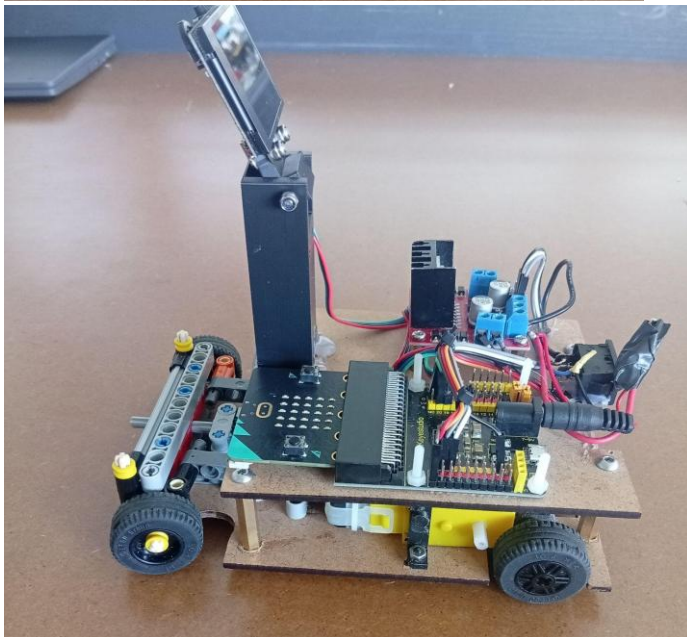
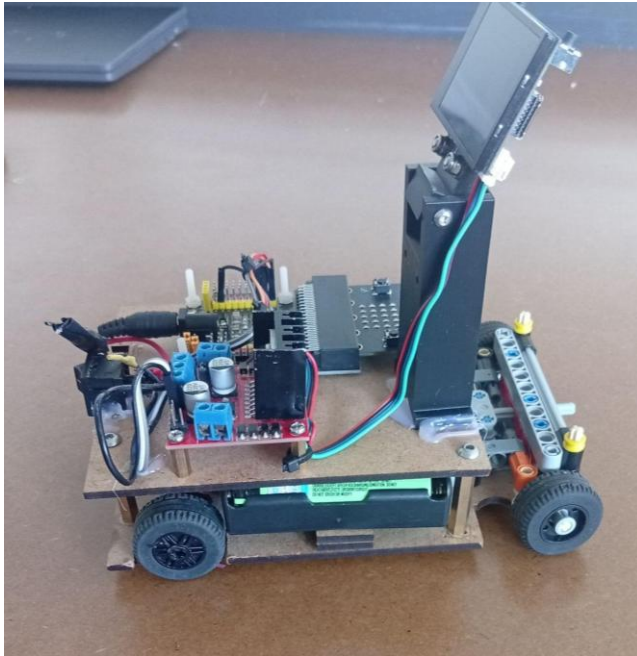
Aceleración: El robot avanza a máxima velocidad (1023).

Obteniendo la coordenada X (izquierda/derecha) y la coordenada Y (arriba/abajo) del objeto en la pantalla de la cámara podemos tomar decisiones.

- Si el objeto está muy a la izquierda ($X < 140$), el servo gira hacia ese lado (ángulo 55).
- Si está muy a la derecha ($X > 180$), gira al otro lado (ángulo 135). Si está centrado entre 140 y 180, pone las ruedas rectas (ángulo 90).
- Si la coordenada Y del objeto es mayor que 200, significa que el objeto está en la parte inferior de la pantalla de la cámara. En visión artificial para robots de suelo, que un objeto "baje" en la pantalla significa que el robot se está acercando a él. Si supera 200, asume que ha tocado la meta, marca carreraTerminada = true, se detiene y muestra una "X" (IconNames.No) en la pantalla LED, indicando el fin del trayecto. Si pierde el objetivo de vista, intenta avanzar recto un momento y luego se rinde.
- Si encuentra una pared (Color 3): El robot frena, gira las ruedas en dirección contraria a donde está el centro de la pared, da marcha atrás, gira al lado opuesto y hace una maniobra en forma de "S" hacia adelante para sortearla.
- Si son obstáculos pequeños (Colores 1 y 2): El robot gira las ruedas, retrocede un poco para hacer espacio y luego endereza.
- El código asigna diferentes giros dependiendo de si es el color 1 (rojo), el color 2 (verde) o el color 3 (negro, pared).
- Cuando no ve nada, pone la dirección completamente recta (ángulo 90) y avanza a una velocidad moderada de 750 buscando constantemente algo en su entorno.

- Fotografías del robot (desde todos los ángulos) y una fotografía del equipo.





- Vídeo demostrativo mostrando el robot funcionando de manera autónoma en las diferentes regiones
- Repositorio público en GitHub
 - El código empleado
 - Documentación del proyecto
 - (Opcional) Modelos 3D o archivos para la fabricación de piezas si las hubiera
 - Un archivo README.md con una descripción clara de la solución, los módulos del código y su relación con los componentes del robot, que recoja: el proceso de desarrollo, las decisiones técnicas adoptadas y los resultados obtenidos.
 - Código comentado y documentado para facilitar su comprensión por parte del jurado